

Fecha: Domingo 8 de setiembre, 2024
Tiempo: 3 horas
Total de Puntos: 100 pts.

- a) La memoria se libera automáticamente. c) Se produce una fuga de memoria.
b) Se produce un error de compilación. d) La memoria se vuelve estática.

4-¿Cuál es una característica de las variables de memoria automática en C++?

- a) Su tamaño se define en tiempo de ejecución.
- b) Su tiempo de vida es el mismo que el de la función en la que se declaran.
- c) Se inicializan solo una vez y persisten a lo largo de la ejecución del programa.
- d) Se reservan usando new.

5-¿Cuál es una característica de las variables definidas como estáticas en C++?

- a) Su tamaño se define en tiempo de ejecución.
- b) Su tiempo de vida es el mismo que el de la función en la que se declaran.
- c) Se inicializan solo una vez y persisten a lo largo de la ejecución del programa.
- d) Se reservan usando new.

6. ¿Cuál de las siguientes afirmaciones sobre la memoria dinámica es correcta?

- a) La memoria se libera una vez finaliza el bloque en el que fue asignada.
- b) Los objetos en memoria dinámica permanecen hasta que el programa termina, independientemente de su liberación.
- c) El uso de delete es necesario para liberar la memoria reservada con new.
- d) Se inicializan solo una vez y persisten a lo largo de la ejecución del programa.

7. Suponiendo ya implementada la clase **parOrdenado**, cuál error tiene el siguiente código:

```
int main() {  
    parOrdenado v[10];  
    v[0].setX(4);  
    v[0].setY(6);  
    cout << v[0].toString();  
    delete v[];  
    return 0;  
}
```

- a) Se hace una incorrecta liberación de memoria.
- b) Se debe utilizar el operador fecha “->” en lugar del operador punto “.”
- c) Se invocan métodos de objetos que no se han creado.
- d) Falta un asterisco al declarar el vector.

8. Dada la clase definida en el recuadro. ¿Cuál es la instrucción que falta en el método **probar()** que permitiría llamar adecuadamente al método **mod()**?

```
class MiClase {  
public:  
    void mod (int &x) {  
        x = x + 1;  
    }  
  
    void probar() {  
        int a=5;  
        int &r = a;  
        _____?  
    }  
};
```

- a) mod (&a);
- b) mod (&r);
- c) mod (*r);
- d) mod (r);

9. En el siguiente código, es cierto que:

```
1 int y = 2;  
2 int &x = y;  
3 int *p = &y;
```

- a) En la segunda línea el operador & es utilizado para acceder a la dirección de x.
- b) En la tercera línea el operador * es utilizado para acceder a la dirección de p.
- c) En la segunda línea el operador & es utilizado para declarar x como un alias o una referencia a y.
- d) En la tercera línea el operador * es utilizado para acceder al contenido de lo que apunta p.

10. En el siguiente código, es cierto que:

```
...
persona *p= new persona();
delete p;
```

- a. El delete libera la memoria asignada a la variable tipo puntero llamada p.
- b. El delete elimina la memoria asignada tanto a p como al objeto persona.
- c. La instrucción delete limpia el puntero de basura.
- d. La instrucción delete elimina la memoria asignada al objeto persona.

11. Analice las siguientes dos líneas de código y determine los tipos de memoria empleados.

```
cliente* ptr;
ptr = new cliente();
```

- a. Únicamente se utiliza memoria dinámica.
- b. Se utiliza tanto memoria automática como dinámica.
- c. Únicamente se utiliza memoria estática.
- d. No se asigna ningún tipo de memoria, pues únicamente se está creando un puntero.

12. Si en una clase **Coleccion** deseamos declarar un vector dinámico de objetos automáticos tipo **Trabajador**, el atributo de la clase para declarar dicho vector debe ser:

- a) **Trabajador *v;**
- b) **Trabajador ***v;**
- c) **Trabajador * v[tam];**
- d) **Trabajador ** v;**

13. Dada la siguiente declaración de la clase **calculo** ¿Cuál opción permite instanciar correctamente un objeto de la clase?

```
class calculo{
private:
    int *x;
    int *y;
public:
    calculo (int *, int *);
};
```

- a) **void main(){**
 int *x;
 int *y;
 calculo (*x, *y);
 }
- b) **void main(){**
 int x= 4;
 int y= 6;
 calculo (x, y);
 }
- c) **void main(){**
 int x= 4;
 int y= 6;
 calculo (*x, *y);
 }
- d) **void main(){**
 int x= 4;
 int y= 6;
 calculo (&x, &y);
 }

14. ¿Cuál es el propósito de la palabra clave **const** utilizada, por ejemplo, en la siguiente instrucción en C++?

```
const float x = 1.14;
```

- a) Crear un alias.
- b) Crear una referencia.
- c) Garantizar que una variable no cambie su valor después de ser inicializada.
- d) Permitir la modificación de una variable a lo largo del programa.

15. ¿Qué significa la siguiente declaración en C++?

```
const persona* ptr = new persona();
```

- a) ptr es un puntero a una persona constante.
- b) ptr es un puntero constante a una persona.
- c) ptr es un puntero que solo puede invocar a métodos constantes de persona.
- d) todas las anteriores.

Comentado [OB1]: Revisar la respuesta correcta.

16. ¿Cuál es el valor final de x ?

```
int probar() {  
    static int x = 10;  
    return ++x;  
}  
  
int main() {  
    int x;  
    x = probar() + 1;  
    x = probar() + 2;  
    x = probar();  
    cout << x;  
    return 0;  
}
```

- a) 11
- b) 10
- c) 13
- d) 15

17. ¿Qué significa que un atributo se declare **static**?

- a) El atributo static no puede cambiar de valor
- b) La memoria asignada al atributo se libera automáticamente cuando se libera la memoria asignada a todos los objetos de la clase
- c) El atributo declarado static corresponde a un único espacio de memoria compartido por todos los objetos de la clase.
- d) Ninguna de las anteriores

II Parte. Complete (21 pts. 1.5 pts. cada ítem)

Dado el siguiente código, complete los espacios con el valor que se muestra con cada instrucción **cout** en el main().

```
#include<iostream>
using namespace std;

class Tripleta {
private:
    int x;
    int y;
    int z;
public:
    Tripleta(int = 2, int = 1, int = 5);
    int getX();
    int getY();
    int getZ();
    void setZ(int);
    void setX(int*);
    void setY(int&);
    void cambio(int*, int&);
};

Tripleta::Tripleta(int x1, int y1, int z1) {
    x = x1;
    y = y1;
    z = z1;
}

int Tripleta::getX() { return x; }
int Tripleta::getY() { return y; }
int Tripleta::getZ() { return z; }
void Tripleta::setX(int* x) { this->x = *x; }
void Tripleta::setY(int& y) { this->y = y; }
void Tripleta::setZ(int z) { this->z = z; }

void Tripleta::cambio(int* p1, int& p2) {
    p2 = 2;
    *p1 = 1;
    this->setX(p1);
    this->setY(p2);
    this->setZ(*p1 + 1);
}

int main() {
    int x = 1, y = 1, z = 1, i = 6, j = 6;
    int* ptr1 = &x;
    int* ptr2 = &y;
    int* ptr3 = &z;
    Tripleta c1(2, 3, 6);
    Tripleta* c2 = new Tripleta;
    *ptr1 = 3 + *ptr3;
    *ptr2 = 5;
    z = 2;
    cout << *ptr1<< endl; // _____ ?
    cout << *ptr2<< endl; // _____ ?
    cout << *ptr3<< endl; // _____ ?
    y = *ptr3;
    *ptr3 = *ptr3 + 2;
    x = 5;
    c1.setX(&x);
    c1.setY(y);
    cout << c1.getX()<< endl; // _____ ?
    cout << c1.getY()<< endl; // _____ ?
    cout << c1.getZ()<< endl; // _____ ?
    cout << x << endl; // _____ ?
    cout << y << endl; // _____ ?
    cout << z << endl; // _____ ?
    c2->cambio(&i, j);
    cout<< c2->getX()<< endl; // _____ ?
    cout<< c2->getY()<< endl; // _____ ?
    cout<< c2->getZ()<< endl; // _____ ?
    cout<< i << endl; // _____ ?
    cout<< j << endl; // _____ ?
    delete c2;
    cin.get();
    return 0;
}
```

III Parte. Desarrollo (45 puntos)

Suponga implementadas las clases **carro** y **propietario**, declaradas a continuación:

```
class carro{
private:
    string placa;
    string annio;
    propietario* elpro;

public:
    carro(string, string, propietario*)
    string getPlaca();
    string getAnnio();
    propietario* getPropietario();
    void setPlaca(string);
    void setAnnio(string);
    void setPropietario(propietario*)
    string toString();
};

class propietario{
private:
    string id;
    string nombre;

public:
    propietario (string, string)
    string getID();
    string getNombre();
    void setID(string);
    void setNombre(string);
    string toString();
};
```

Implemente una clase **coleccion** la cual permita administrar un arreglo unidimensional (vector) dinámico de **n** carros creados con asignación dinámica de memoria. Incluya lo siguiente:

- **Declaración de los Atributos** de la clase **coleccion**. (5pts)
- **Constructor**. Este método inicializa adecuadamente todos los atributos. (6pts)
- **Destructor**. Este método libera adecuadamente toda la memoria asignada, incluida la memoria asignada a cada objeto carro incluido en la **coleccion**. (6pts)
- Método **insertar**. Este método recibe un puntero a un **carro** y una **posición**, e inserta la dirección recibida como parámetro en la **posición** correspondiente del arreglo. Si la **posición** está ocupada y el vector no está lleno, se debe hacer un corrimiento de los elementos del arreglo que permita insertar en la **posición**. (5pts)
- Método **toString**. Retorna una hilera que permite mostrar los datos de cada uno de los carros asociados al arreglo, incluyendo la información completa de su propietario. (6pts)
- Método **eliminar**. Este método recibe un id de propietario y elimina todos los carros de dicho propietario. Se deben realizar los corrimientos requeridos en el arreglo. (10pts)
- Método **buscarMasAntiguo**. Este método debe retornar la dirección del objeto carro más antiguo asociado a la coleccion. (7pts)